5 - Generadores 🧩

Tabla de contenidos

1 Introducción 1
2 Funciones generadoras 1
2.1 Ejemplo 1: Secuencia de números naturales 4
2.2 Ejemplo 2: Secuencia de Fibonacci 6
2.3 Ejemplo 3: Promedio acumulado 7
3 Expresiones generadoras 7
4 Iterables e iteradores 9

1 Introducción

En este capítulo vamos a introducirnos en los generadores, tanto en las funciones como en las expresiones generadoras. A diferencia de las funciones regulares, que devuelven un resultado con `return`, los generadores no devuelven un único resultado, sino que van entregando valores de a uno a medida que se lo solicita. Cada vez que se entrega un valor, la ejecución queda en pausa y se conserva el estado de las variables, de modo que puede reanudarse más adelante. De este modo, los generadores resultan ideales para definir iteradores y trabajar con grandes volúmenes de datos sin necesidad de almacenarlos al mismo tiempo en memoria.

2 Funciones generadoras

Una función generadora se define igual que una función común con `def`, pero en lugar de devolver un valor con `return`, lo hace con `yield`.

Cuando se ejecuta una función generadora, no se ejecuta el código en su cuerpo de manera inmediata ni se obtiene un resultado. En cambio, se obtiene un **generador** que luego puede **entregar** valores.

```
def gen():
    yield "¡Resultado!"

g = gen()
g
```

```
<generator object gen at 0x7f90c8738ca0>
```

Como los generadores son **iteradores** (ver Iterables e iteradores), se puede usar `next` para obtener el siguiente valor de manera manual:

```
next(g)
```

```
'¡Resultado!'
```

Este primer ejemplo es demasiado simple para apreciar la verdadera utilidad de los generadores. Si solo necesitáramos devolver un único valor, bastaría con usar una función común.

La ventaja de los generadores está en que pueden **entregar varios valores de a uno**, a medida que se los solicita, mientras conservan el estado de las variables.

Veamos ahora una segunda función generadora, esta vez con dos instrucciones `yield` en lugar de una. En la primera llamada a `next`, obtenemos `"Primer resultado"`.

```
def gen():
    yield "Primer resultado"
    yield "Segundo resultado"

g = gen()
next(g)
```

```
'Primer resultado'
```

Y en la segunda llamada a `next` el generador entrega el segundo valor: `"Segundo resultado"`.

```
next(g)
```

```
'Segundo resultado'
```

En la primera llamada a `next`, la función se ejecuta hasta llegar a la primera instrucción `yield`. Allí el generador devuelve un valor y suspende su ejecución. Con la segunda llamada, la ejecución se reanuda desde ese punto y continúa hasta encontrar el siguiente `yield`, entregando otro valor.

Ahora bien, ¿qué ocurre si llamamos a `next` cuando el generador ya entregó todos los valores disponibles?

```
next(g)
```

```
    next(g)
    ~~~~^^^
StopIteration
```

Una vez que un generador se agota, cualquier llamada adicional a `next` elevará la excepción `StopIteration`, que señala que ya no quedan valores por producir.

Para observar con más detalle cómo funciona la ejecución y suspensión en los generadores, vamos a implementar una función que mantiene el estado de una variable numérica e imprime un mensaje justo antes de cada `yield`.

```
def generador(x):
    print("Recibí el valor", x)

    x = x + 18
    print("Entrego el valor", x)
    yield x

    x = x - 5
    print("Esto una entrega siguiente, devuelvo el valor", x)
    yield x

    print("Este mensaje está bien al final")

g = generador(7)
```

Como se puede observar, la ejecución de la función generadora no imprimió ningún mensaje, ya que esto no ejecuta el cuerpo de la función. Recién al pedir el primer valor se ejecutan los dos `print` previos al primer `yield`. Además, el valor inicial `7` se incrementa en `18` y luego es devuelto.

```
next(g)
```

```
Recibí el valor 7
Entrego el valor 25
25
```

En la segunda llamada a `next(g)` se imprime un mensaje y se entrega el valor `25 - 5 = 20`. Esto muestra que el generador conserva el estado de las variables: en lugar de usar el valor original de `x`, utiliza el valor actualizado en la entrega anterior.

```
next(g)
```

```
Esto una entrega siguiente, devuelvo el valor 20
20
```

Sin embargo, el `print` al final, debajo del último `yield`, aún no se ejecutó. Para eso, usamos `next(g)` nuevamente.

```
next(g)
```

```
Este mensaje está bien al final

    next(g)
    ~~~~^^^
StopIteration
```

Como no hay ningún otro valor por entregar, se imprime el mensaje y luego se obtiene la excepción `StopIteration`.

2.1 Ejemplo 1: Secuencia de números naturales

Los generadores también permiten crear secuencias infinitas. Para ello, basta con escribir un bucle infinito dentro de la función generadora. Esto no representa un problema, ya que el generador produce un valor a la vez, únicamente cuando se le solicita.

```
def numeros_naturales():
    n = 0
    while True:
        yield n
        n = n + 1

secuencia = numeros_naturales()
```

Luego, pedimos los valores de a uno:

```
print(next(secuencia))
print(next(secuencia))
print(next(secuencia))
print(next(secuencia))
print(next(secuencia))
```

```
0
1
2
3
4
```

Vale la pena notar que un generador no tiene longitud, ya que podría ser infinito, ni permite acceder a sus elementos por índice. Solo sabe cómo producir el próximo valor, sin conocer de antemano cuántos quedan por generar.

```
len(secuencia)
```

```
    len(secuencia)
    ~~~^^^^^^^^^^
TypeError: object of type 'generator' has no len()
```

```
secuencia[0]
```

```
    secuencia[0]
    ~~~~~~~~~^^^
TypeError: 'generator' object is not subscriptable
```

Como los generadores son iteradores, podemos recorrerlos con un bucle `for`. En el caso de secuencias infinitas, es necesario usar un `break` para evitar que el bucle nunca termine.

```
i = 0
for n in secuencia:
    print(n)
    i += 1
    if i >= 5:
        break
```

```
5
6
7
8
9
```

Si nuestro único objetivo es recorrer los elementos del generador, podemos inicializarlo directamente en el bucle `for`.

```
for n in numeros_naturales():
    print(n)
    if n >= 7:
        break
```

```
0
1
2
3
4
5
6
7
```

2.2 Ejemplo 2: Secuencia de Fibonacci

Las secuencias infinitas no se limitan a los números naturales. Como los generadores conservan el estado de las variables dentro de la función, también pueden usarse para producir otras secuencias, como la de Fibonacci.

$$F_n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ F_{n-1} + F_{n-2} & \text{si } n \geq 2 \end{cases}$$

```
def fibonacci():
    a = 0
    b = 1
    while True:
        yield a
        a, b = b, a + b
```

Si queremos los primeros 10 números de la secuencia, podemos utilizar un bucle `for` que ejecuta `next(g)` 10 veces seguidas.

```
g = fibonacci()
for _ in range(10):
    print(next(g))
```

```
0
1
1
2
3
5
8
13
21
34
```

Al volver a pedir un nuevo valor a nuestro generador, este continúa avanzando en la secuencia de Fibonacci.

```
next(g)
```

```
55
```

```
next(g)
```

```
89
```

2.3 Ejemplo 3: Promedio acumulado

En este ejemplo se muestra un generador que procesa una secuencia numérica y va devolviendo el promedio acumulado a medida que avanza.

Como los valores se producen bajo demanda, en memoria solo se conserva la secuencia original y el último promedio calculado.

```python
def promedio_acumulado(numeros):
    numerador = 0
    for i, numero in enumerate(numeros):
        numerador += numero
        yield numerador / (i + 1)

valores = [2, 4, 9, 1, 7, 11] # Supongamos una lista muy grande de números

for m in promedio_acumulado(valores):
    print(f"Promedio acumulado: {m:.2f}")
```

```
Promedio acumulado: 2.00
Promedio acumulado: 3.00
Promedio acumulado: 5.00
Promedio acumulado: 4.00
Promedio acumulado: 4.60
Promedio acumulado: 5.67
```

3 Expresiones generadoras

Las expresiones generadoras, del inglés *generator expressions*, proveen una manera concisa para construir generadores. Se parecen a las *list comprehensions*, pero usan paréntesis en vez de corchetes.

Supongamos una lista de números cualquiera y que usamos una *list comprehension* para obtener el triple de cada número.

```python
numeros = [3, 14, 2, 7, 1, 28]
triples = [n * 3 for n in numeros]

print(numeros)
print(triples)
```

```
[3, 14, 2, 7, 1, 28]
[9, 42, 6, 21, 3, 84]
```

La expresión generadora equivalente es la siguiente:

```
triples = (n * 3 for n in numeros)
triples
```

```
<generator object <genexpr> at 0x7f90c87702b0>
```

Como todo generador, implementa la estrategia de evaluación perezosa. Esto quiere decir que el triple de cada número se calcula justo en el momento en que se solicita, no antes.

Así, podemos obtener los valores mediante un bucle:

```
for n in triples:
    print(n)
```

```
9
42
6
21
3
84
```

Un generador creado con una expresión generadora es equivalente a uno definido con una función generadora. En ambos casos, si se intenta obtener un valor de un generador ya agotado, se producirá un error.

```
next(triples)
```

```
    next(triples)
    ~~~~^^^^^^^^^
StopIteration
```

Y al intentar obtener una lista a partir de un generador agotado, obtendremos una lista vacía.

```
list(triples)
```

```
[]
```

Además de ser perezosos, los generadores son de único uso. Sus valores se generan a medida que se solicitan y no se guardan en memoria, de modo que, una vez consumidos, no es posible volver a iterarlos.

Esta aparente limitación es en realidad una ventaja. A diferencia de una lista, que construye y guarda todos sus elementos en memoria, un generador solo define una receta para producirlos

cuando se necesiten. En el siguiente ejemplo se muestra cómo esto impacta en el consumo de memoria frente a una lista.

```
import sys

# Enteros divisibles por 3 o 5 entre 1 y 10,000,000
lista = [n for n in range(1, 10_000_001) if n % 3 == 0 or n % 5 == 0]
genexpr = (n for n in range(1, 10_000_001) if n % 3 == 0 or n % 5 == 0)

print(sys.getsizeof(lista))   # bytes
print(sys.getsizeof(genexpr)) # bytes
```

```
39064728
200
```

Y a partir de ambos objetos se puede computar, por ejemplo, la suma.

```
sum(lista), sum(genexpr)
```

```
(23333341666668, 23333341666668)
```

En resumen, mientras que una **lista** es una **colección de valores**, un **generador** es una **receta para producir valores**.

4 Iterables e iteradores

A lo largo de este capítulo dijimos varias veces que **los generadores son iteradores**, aunque todavía no definimos con precisión qué significa eso.

Lo que sí sabemos es que un objeto es **iterable** cuando puede recorrerse con un bucle `for`. En Python, las listas, las cadenas y los diccionarios son ejemplos de objetos iterables, por lo que los siguientes bloques de código funcionan sin problemas:

```
for i in [10, 55, 2]:
    print(i + 5)

for c in "palabras":
    print(c.upper())

for k in {"nombre": "Juan", "apellido": "Pérez"}:
    print(k)
```

Como ya vimos que una lista se puede recorrer con un bucle, podríamos preguntarnos si también es posible usar la función `next` para obtener su siguiente elemento.

```
nums = [-10, 0, 10]
```

```
next(nums)
```

```
    next(nums)
    ~~~~^^^^^^
TypeError: 'list' object is not an iterator
```

Sin embargo, al hacerlo obtenemos un `TypeError` que indica que la lista no es un **iterador**. Lo mismo ocurre si intentamos usar `next` directamente con una cadena o un diccionario.

```
next("palabra")
```

```
    next("palabra")
    ~~~~^^^^^^^^^^^
TypeError: 'str' object is not an iterator
```

```
next({"nombre": "Juan", "apellido": "Pérez"})
```

```
    next({"nombre": "Juan", "apellido": "Pérez"})
    ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'dict' object is not an iterator
```

El error que aparece al usar `next` sobre una lista, una cadena o un diccionario muestra que no basta con que un objeto sea iterable para poder aplicarle `next` directamente.

Lo que sucede, es que, en realidad, nuestra definición inicial de iterable era incompleta: un objeto es **iterable** cuando puede generar un **iterador** a partir de él.

Luego, es el iterador que conoce cómo producir los valores uno a uno y, por eso, es sobre el iterador (y no sobre el iterable) que Python puede aplicar `next` para avanzar en la secuencia.

Para crear un iterador a partir de un iterable usamos `iter`.

```
iterador = iter(nums)
iterador
```

```
<list_iterator at 0x7f90c87337f0>
```

Y ahora sí es posible avanzar a través de los elementos de la lista original:

```
next(iterador)
```

```
-10
```

```
next(iterador)
```

```
0
```

```
next(iterador)
```

```
10
```

```
next(iterador)
```

```
    next(iterador)
    ~~~~^^^^^^^^^^
StopIteration
```

Por último, vale la pena señalar que los iteradores solo pueden construirse a partir de objetos **iterables**. Por ejemplo, un número entero no es iterable, por lo que no es posible obtener un iterador a partir de él.

```
iter(10)
```

```
    iter(10)
    ~~~~^^^^
TypeError: 'int' object is not iterable
```

En resumen, en Python **solo se puede iterar sobre iteradores**. Un objeto es **iterable** cuando puede generar un iterador a partir de él, y es este último el que sabe cómo devolver los elementos uno a uno mediante la función `next`. Cuando ya no quedan más valores por producir, el iterador eleva la excepción `StopIteration`.

Los **generadores son un caso particular de iteradores**: producen sus valores bajo demanda y mantienen el estado entre llamadas.

Finalmente, al usar un bucle `for` con un iterable, todo este mecanismo ocurre de forma automática: Python crea el iterador por nosotros y se encarga de avanzar en la secuencia hasta agotarla.